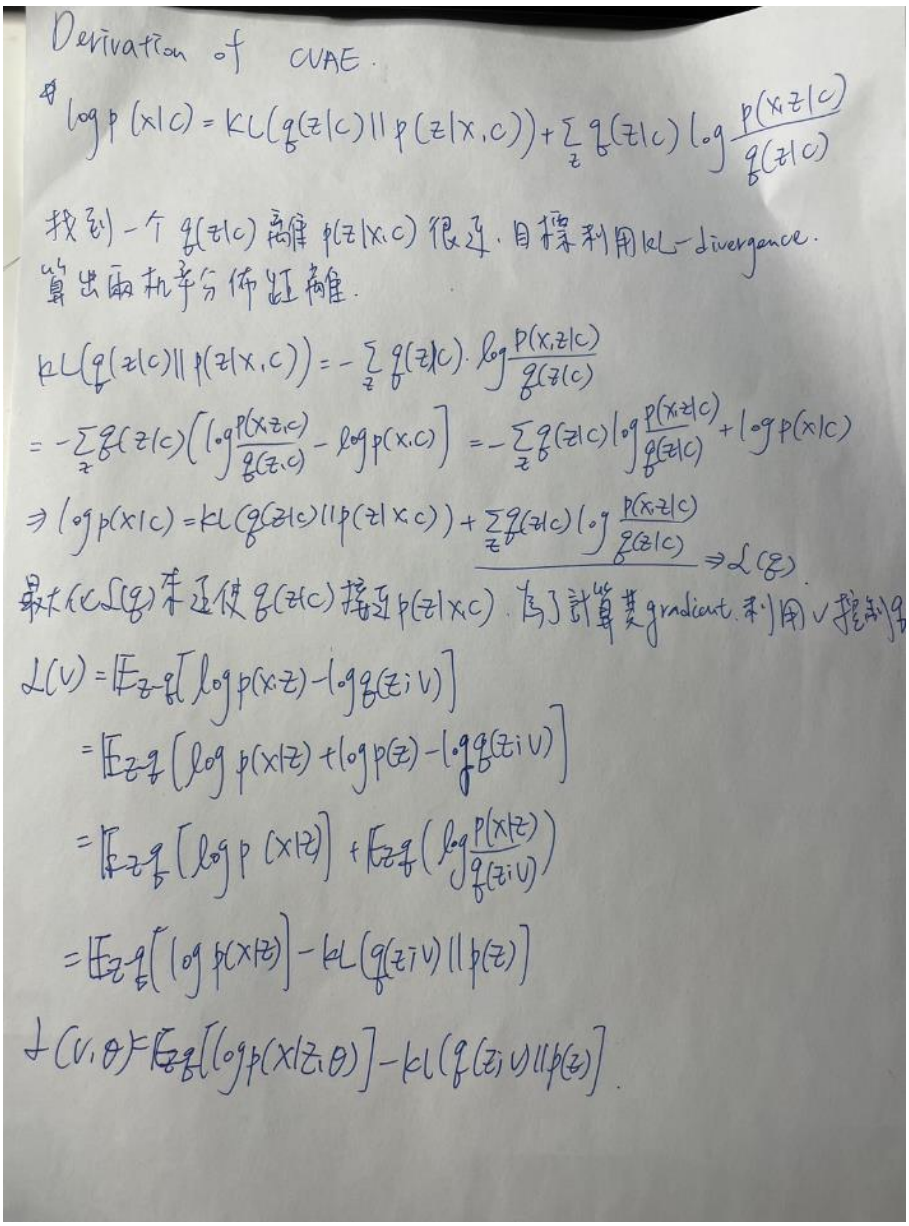
Lab5 - Conditional VAE For Video Prediction

- ## Introduction

  這個作業我們要去實作出一個 CVAR 預測影像方面的 Lab，主要就是我們會去利用前一個 frame 當作 encoder 的輸入，使他生成 latent vector，並且也會從 fixed prior 中 sample 一個 zt，最終我們將來自 encoder 的輸出和 zt 與動作和位置（條件）一起作為 decoder 的輸入，並期望輸出應該是下一個的 frame。

- ## Derivation of CVAE



- **Implementation details**

(1) Describe how you implement your model

Dataloader

一開始先把資料的路徑給選好，接著讀取且合併裡面的兩個 csv (actions、endeffector_positions) 回傳給 cond，而 seq 會讀入圖片的資訊。

```
class bair_robot_pushing_dataset(Dataset):
    def __init__(self, args, mode='train', transform=default_transform):
        assert mode == 'train' or mode == 'test' or mode == 'validate'
        self.root = '{}/{}'.format(args.data_root, mode)
        # self.seq_len = max(args.n_past + args.n_future, args.n_eval)
        self.mode = mode
        if mode == 'train':
            self.ordered = False
        else:
            self.ordered = True
        self.seq_len = args.n_past + args.n_future
        self.transform = transform
        self.dirs = []
        for dir1 in os.listdir(self.root):
            for dir2 in os.listdir(os.path.join(self.root, dir1)):
                self.dirs.append(os.path.join(self.root, dir1, dir2))

        self.seed_is_set = False
        self.d = 0
        self.cur_dir = self.dirs[0]
```

```
def get_csv(self):
    with open('{}/actions.csv'.format(self.cur_dir), newline='') as csvfile:
        rows = csv.reader(csvfile)
        actions = []
        for i, row in enumerate(rows):
            if i == self.seq_len:
                break
            action = [float(value) for value in row]
            actions.append(torch.tensor(action))

        actions = torch.stack(actions)

    with open('{}/endeffector_positions.csv'.format(self.cur_dir), newline='') as csvfile:
        rows = csv.reader(csvfile)
        positions = []
        for i, row in enumerate(rows):
            if i == self.seq_len:
                break
            position = [float(value) for value in row]
            positions.append(torch.tensor(position))
        positions = torch.stack(positions)

    condition = torch.cat((actions, positions), axis=1)

    return condition

def __getitem__(self, index):
    self.set_seed(index)
    seq = self.get_seq()
    cond =  self.get_csv()
    return seq, cond
```

Encoder

利用 vgg64 encoder 來 output 一個經過壓縮編碼的 latent vector，並且她還返回了在中間層的圖，這部分會用在後面 decoder 時。

```
def forward(self, input):
    h1 = self.c1(input) # 64 -> 32
    h2 = self.c2(self.mp(h1)) # 32 -> 16
    h3 = self.c3(self.mp(h2)) # 16 -> 8
    h4 = self.c4(self.mp(h3)) # 8 -> 4
    h5 = self.c5(self.mp(h4)) # 4 -> 1
    return h5.view(-1, self.dim), [h1, h2, h3, h4]
```

```python
class vgg_encoder(nn.Module):
    def __init__(self, dim):
        super(vgg_encoder, self).__init__()
        self.dim = dim
        # 64 x 64
        self.c1 = nn.Sequential(
                vgg_layer(3, 64),
                vgg_layer(64, 64),
                )
        # 32 x 32
        self.c2 = nn.Sequential(
                vgg_layer(64, 128),
                vgg_layer(128, 128),
                )
        # 16 x 16
        self.c3 = nn.Sequential(
                vgg_layer(128, 256),
                vgg_layer(256, 256),
                vgg_layer(256, 256),
                )
        # 8 x 8
        self.c4 = nn.Sequential(
                vgg_layer(256, 512),
                vgg_layer(512, 512),
                vgg_layer(512, 512),
                )
        # 4 x 4
        self.c5 = nn.Sequential(
                nn.Conv2d(512, dim, 4, 1, 0),
                nn.BatchNorm2d(dim),
                nn.Tanh()
                )
        self.mp = nn.MaxPool2d(kernel_size=2, stride=2, padding=0)
```

lstm

首先他會先將 encoder output embed 成所需要的大小，用來給之後的 decoder fixed size。

```python
class lstm(nn.Module):
    def __init__(self, input_size, output_size, hidden_size, n_layers, batch_size, device):
        super(lstm, self).__init__()
        self.device = device
        self.input_size = input_size
        self.output_size = output_size
        self.hidden_size = hidden_size
        self.batch_size = batch_size
        self.n_layers = n_layers
        self.embed = nn.Linear(input_size, hidden_size)
        self.lstm = nn.ModuleList([nn.LSTMCell(hidden_size, hidden_size) for i in range(self.n_layers)])
        self.output = nn.Sequential(
                nn.Linear(hidden_size, output_size),
                nn.BatchNorm1d(output_size),
                nn.Tanh())
        self.hidden = self.init_hidden()

    def init_hidden(self):
        hidden = []
        for _ in range(self.n_layers):
            hidden.append((Variable(torch.zeros(self.batch_size, self.hidden_size).to(self.device)),
                           Variable(torch.zeros(self.batch_size, self.hidden_size).to(self.device))))
        return hidden
```

```
def forward(self, input):
    embedded = self.embed(input)
    h_in = embedded
    for i in range(self.n_layers):
        self.hidden[i] = self.lstm[i](h_in, self.hidden[i])
        h_in = self.hidden[i][0]

    return self.output(h_in)
```

Gaussian lstm

用來建構一個 gaussian lstm，來回傳 sample z 用以加入 encoder output 中。

```
class gaussian_lstm(nn.Module):
    def __init__(self, input_size, output_size, hidden_size, n_layers, batch_size, device):
        super(gaussian_lstm, self).__init__()
        self.device = device
        self.input_size = input_size
        self.output_size = output_size
        self.hidden_size = hidden_size
        self.n_layers = n_layers
        self.batch_size = batch_size
        self.embed = nn.Linear(input_size, hidden_size)
        self.lstm = nn.ModuleList([nn.LSTMCell(hidden_size, hidden_size) for i in range(self.n_layers)])
        self.mu_net = nn.Linear(hidden_size, output_size)
        self.logvar_net = nn.Linear(hidden_size, output_size)
        self.hidden = self.init_hidden()

    def init_hidden(self):
        hidden = []
        for _ in range(self.n_layers):
            hidden.append((Variable(torch.zeros(self.batch_size, self.hidden_size).to(self.device)),
                           Variable(torch.zeros(self.batch_size, self.hidden_size).to(self.device))))
        return hidden
```

```
def forward(self, input):
    embedded = self.embed(input)
    h_in = embedded
    for i in range(self.n_layers):
        self.hidden[i] = self.lstm[i](h_in, self.hidden[i])
        h_in = self.hidden[i][0]
    mu = self.mu_net(h_in)
    logvar = self.logvar_net(h_in)
    z = self.reparameterize(mu, logvar)
    return z, mu, logvar
```

Reparameterization trick

我們利用 mu 與 var 去得到 gaussian distribution z 時，這邊必須要用 log 的 variance

```
def reparameterize(self, mu, logvar):
    vector_size = logvar.size()
    eps = Variable(torch.FloatTensor(vector_size).normal_()).to(self.device)
    std = logvar.mul(0.5).exp_()
    return eps.mul(std).add_(mu)
```

Vgg decoder

把剛剛 sample part 的 output 和 encoder 的中間 4 個中間層來做 decoder 去預測產生下一個圖

```
class vgg_decoder(nn.Module):
    def __init__(self, dim):
        super(vgg_decoder, self).__init__()
        self.dim = dim
        # 1 x 1 -> 4 x 4
        self.upc1 = nn.Sequential(
                nn.ConvTranspose2d(dim, 512, 4, 1, 0),
                nn.BatchNorm2d(512),
                nn.LeakyReLU(0.2, inplace=True)
                )
        # 8 x 8
        self.upc2 = nn.Sequential(
                vgg_layer(512*2, 512),
                vgg_layer(512, 512),
                vgg_layer(512, 256)
                )
        # 16 x 16
        self.upc3 = nn.Sequential(
                vgg_layer(256*2, 256),
                vgg_layer(256, 256),
                vgg_layer(256, 128)
                )
        # 32 x 32
        self.upc4 = nn.Sequential(
                vgg_layer(128*2, 128),
                vgg_layer(128, 64)
                )
        # 64 x 64
        self.upc5 = nn.Sequential(
                vgg_layer(64*2, 64),
                nn.ConvTranspose2d(64, 3, 3, 1, 1),
                nn.Sigmoid()
                )
        self.up = nn.UpsamplingNearest2d(scale_factor=2)
```

```
def forward(self, input):
    vec, skip = input
    d1 = self.upc1(vec.view(-1, self.dim, 1, 1)) # 1 -> 4
    up1 = self.up(d1) # 4 -> 8
    d2 = self.upc2(torch.cat([up1, skip[3]], 1)) # 8 x 8
    up2 = self.up(d2) # 8 -> 16
    d3 = self.upc3(torch.cat([up2, skip[2]], 1)) # 16 x 16
    up3 = self.up(d3) # 8 -> 32
    d4 = self.upc4(torch.cat([up3, skip[1]], 1)) # 32 x 32
    up4 = self.up(d4) # 32 -> 64
    output = self.upc5(torch.cat([up4, skip[0]], 1)) # 64 x 64
    return output
```

(2) Describe the teacher forcing

使用 teacher forcing 最主要的想法在於，我們直接將正確的答案加入倒 encoder 去，直接影響下一個的預測，而非是生成器的輸出作為下一刻的輸入，而優點就顯而易見，他直接加速模型收斂，因為模型就可以直接看到正確的分布應該為何，也可以減少其 loss，而缺點可能就在於實際應用時，會有訓練與預測不一致的問題，因為我們在真實情況下一定是，拿之前的當作輸入產生新樣本，而非直接看到答案，這樣就可能太相信前者，導致模型沒有 exploration。所以在使用時，我有把它 balance 一下，採用動態的 teacher forcing ratio。

```
if epoch >= args.tfr_start_decay_epoch:
    ### Update teacher forcing ratio ###
    slip = 1.0 / (args.niter - args.tfr_start_decay_epoch)
    tfr = 1.0 - (epoch - args.tfr_start_decay_epoch) * slip
    args.tfr = min(1, max(0, tfr))
```

• Results and discussion

(1) Show your results of video prediction

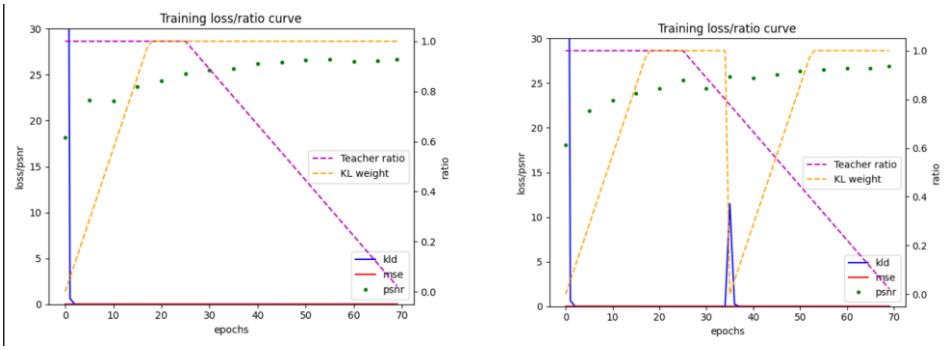
(a) Make videos or gif images for test result (select one sequence)

此為 gif 檔的截圖，最左邊為 ground truth，其餘為預測產生。

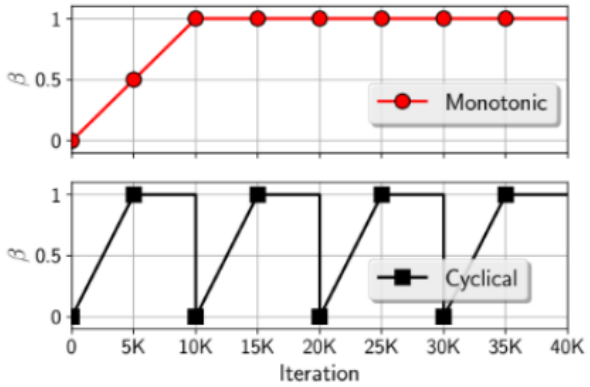
(b) Output the prediction at each time step (select one sequence)

前兩項是 ground truth，後面為其預測產生的 10 張圖。

(2) Plot the KL loss and PSNR curves during training



這個是使用下圖兩個不同的 KL annealing schedule 所產生的，左邊為 Monotonic，右邊則是 Cyclical。



where β is the trade-off between minimizing frame prediction error and fitting the prior.

(3) Discuss the results according to your setting of teacher forcing ratio, KL weight, and learning rate.

如果一開始沒有使用到 teacher forcing 的話，會使得他 loss 非常大，而且很難收斂，所以這有點像我們在做 lab6 的 warmup，我先將他設定前 25 個 epoch 有個老師指導她走路，所以將 teacher forcing 設為 0，並且這之後就直接將剩下的 epoch 的 teacher forcing 慢慢降至到 0，這使得我的 psnr 很快就可以達到不錯的數值。